

WINDOWS KERNEL TELEMETRY & DETECTION TECHNIQUES

CHEAT SHEET

PowerShell Commands

Listing all registered ETW providers on the system, displaying their **names** and corresponding **GUIDs**:

```
PS C:\htb> logman query providers
```

```
PS C:\htb> logman query providers | findstr /I "Microsoft-Windows-Kernel-Process"
```

Displaying detailed information about a specific ETW provider, including its GUID and its available keywords:

```
PS C:\htb> logman query providers Microsoft-Windows-Threat-Intelligence
```

List the providers a process is capable of sending data to:

```
PS C:\htb> logman query providers -pid 3632
```

Displaying all currently active **Event Trace Sessions (ETS)** on the system in addition to their **state** (Running or not):

```
PS C:\htb> logman query -ets
```

Retrieving details about a specific active Event Trace Session, including its **associated providers**:

```
PS C:\htb> logman query "EventLog-System" -ets
```

Loading the **mimikatz.sys** driver via **mimikatz.exe (!+)** and displaying the various kernel notification callback functions registered on the system:

```
mimikatz # !+
```

```
mimikatz # !notifProcess
```

```
mimikatz # !notifThread
```

```
mimikatz # !notifImage
```

```
mimikatz # !notifReg
```

WTF-WFP PowerShell Module

```
Install-Module -Name NtObjectManager
```

```
Install-Module wtf-wfp
```

Retrieving information about available WFP layers:

```
PS C:\htb> Get-FwLayer
```

Displaying Sub Layers and their respective weight:

```
PS C:\htb> Get-FwSubLayer | Sort-Object Weight -Desc | Select-Object KeyName,Weight
```

Displaying registered Filters:

```
PS C:\htb> Get-FwFilter
```

Counting the number of Filters per Sub Layer:

```
PS C:\htb> Get-FwFilter | Group-Object -Property SubLayerKeyName | Select-Object Name,Count
```

Counting the number of Filters per layer:

```
PS C:\htb> Get-FwFilter | Group-Object -Property LayerKeyName | Select-Object Name,Count
```

Counting the number of Filters for each Action Type:

```
PS C:\htb> Get-FwFilter | Group-Object -Property ActionType | Select-Object Name,Count
```

Enumerating registered callouts:

```
PS C:\htb> Get-FwCallout
```

WinDbg Kernel Debugging Commands

Displaying WMI event trace structures for all trace sessions, allowing to retrieve each session's **identifier**, its **kernel structure address** and its **name**:

```
!wmitrace.strdump
```

Dereferencing the **Consumers** linked list of the specified **WMI_LOGGER_CONTEXT** structure, revealing consumers currently consuming events from that ETW logger session whose kernel representation add

```
dx &((nt!_WMI_LOGGER_CONTEXT*)0xFFFFC6896C5C3040)->Consumers
```

Traversing the linked list of ETW real-time consumers starting at the specified address, extracting each **ETW_REALTIME_CONSUMER** structure by following its **Links** field, and saving the consumers list withi

```
dx @$consumers = Debugger.Utility.Collections.FromListEntry(*(nt!_LIST_ENTRY*)0xffffc6896c5c3188, "nt!_ETW_REALTIME_CONSUMER", "Links")
```

Displaying the image **file names** of the processes registered as **consumers**:

```
dx @$consumers.Select(p => (char*)p.ProcessObject->ImageFileName)
```

Enumerates each active trace session (identified by a **Logger Id**) and lists the ETW providers currently enabled for it:

```
dx @$silo=(nt!_ETW_SILODRIVERSTATE*)*((void*)&nt!EtwHostSiloState)
```

```
dx -r2 ((nt!_ETW_HASH_BUCKET *) [64])@$silo->EtwpGuidHashTable->SelectMany(b => b.ListHead.SelectMany(x => Debugger.Utility.Collections.FromListEntry(x, "ntkrnlmp!_ETW_GUID_ENTRY")
```

Listing global handles associated with ETW kernel providers.

```
x nt!Etw*Handle
```

Showing the **ETW_REG_ENTRY** for the ETW-TI kernel provider via **nt!EtwThreatIntProvRegHandle**:

```
dt _ETW_REG_ENTRY poi(nt!EtwThreatIntProvRegHandle)
```

Displaying the GUID of the ETW-TI kernel provider:

```
dx (*( _ETW_REG_ENTRY*)&nt!EtwKernelProvRegHandle)->GuidEntry->Guid
```

Setting a "callback" breakpoint on **nt!EtwTiLogAllocExecVm** that logs the executable memory allocation address (**@r8**) and the caller's image name (**@rcx+0x5a8**) each time an executable region is alloc

```
bp nt!EtwTiLogAllocExecVm ".printf \"Executable Memory Alloc at 0x%p from %ma\\n\", @r8, @rcx+0x5a8;g"
```

Displaying the address and the value of the **ProviderEnableInfo** member within the **ETW_GUID_ENTRY** structure located here at **0xfffffad01d3898550**:

```
dx &(( _ETW_GUID_ENTRY*)0xfffffad01`d3898550)->ProviderEnableInfo
```

Displays the Protection field of the **_EPROCESS** structure at address **0xfffffd014a936080**, which indicates the process protection **level** and **type**:

```
dx ((_EPROCESS*)0xfffffd014a936080)->Protection
```

Retrieves the **root** of the VAD tree for a process:

```
dx ((_EPROCESS*)0xfffff9308cd3830c0)->VadRoot
```

Dumping the VAD tree starting at the root whose address is **0xfffff9308d1171140**.

```
!vad 0xfffff9308d1171140
```

Showing flags related to **protections** and **properties** for a memory range described by a VAD:

```
dx ((_MMVAD*)0xfffff9308d14319c0)
```

```
dx ((_MMVAD*)0xfffff9308d14319c0)->Core->u->VadFlags
```

Retrieving the Virtual Page Number (VPN) range of a VAD.

```
dx ((_MMVAD*)0xfffff9308d14319c0)->Core.StartingVpn
```

```
dx ((_MMVAD*)0xfffff9308d14319c0)->Core.EndingVpn
```

Extracts the **file path** associated with a mapped image:

```
dx ((_FILE_OBJECT*)(( _MMVAD*)0xfffff9308d14319c0)->Subsection->ControlArea->FilePointer.Object & ~0xF)->FileName
```

Displays metadata about the mapped image, including its **characteristics**, **entry point**, and **stack size**:

```
dx ((_MMVAD*)0xfffff9308d14319c0)->Subsection->ControlArea->Segment->u2.ImageInformation->ExportedImageInformation
```

Enumerating process creation callback routines and displaying the symbol name for a function pointer with **.printf**:

```
dx ((void**[0x40])&nt!PspCreateProcessNotifyRoutine).Where(c => c != 0).Select(c => *(void**)((c & ~0xF) + 8))
.printf "%y\\n", 0xfffff8051b926dd0
```

Creating a list of all active processes and filtering the ones that have missing **SeAuditProcessCreationInfo->ImageFileName->Name**:

```
dx @$processList = Debugger.Utility.Collections.FromListEntry(*(nt!_LIST_ENTRY*)&(nt!PsActiveProcessHead), "nt!_EPROCESS", "ActiveProcessLinks")
```

```
dx @$processList.Where(p => p.SeAuditProcessCreationInfo->ImageFileName->Name.Length == 0).Select(p => (int)p.UniqueProcessId)
```

Checking which security mitigations are enabled for the process whose kernel object is at **0xfffffe70be8fae300**:

```
dx ((_EPROCESS*)0xfffffe70be8fae300)->MitigationFlags2Values
```

Listing major functions of **ntfs.sys** and **npfs.sys**:

```
!drvobj \FileSystem\Ntfs 7
```

```
!drvobj \FileSystem\Npfs 7
```

Displaying all instances of loaded minifilter drivers and their respective **altitude**:

```
!fltkd.filters
```

Dumping information about the minifilter instance whose address is ffffa70f0adb6010:

```
dt _FLT_INSTANCE ffffa70f0adb6010
```

Listing **all callbacks** for the filter instance:

```
dx ((_FLT_INSTANCE*)0xfffffa70f0adb6010)->CallbackNodes->Where(cn => cn != 0x0)
```

Listing only **pre-operation callbacks** for the filter instance:

```
dx ((_FLT_INSTANCE*)0xfffffa70f0adb6010)->CallbackNodes->Where(cn => cn != 0x0).Select(c => c->PreOperation)
```

Retrieving detailed information about specific filter operation handlers, including the associated major function, the pre-operation callback, and the post-operation callback:

```
dx -r2 ((_FLT_INSTANCE*)0xfffffa70f0adb6010)->CallbackNodes->Where(cn => cn != 0x0).Select(c => c->Instance->Filter->Operations)[15]
```

Displaying the security context of the current thread (@\$curthread):

```
dx -r2 @$curthread.KernelObject.ClientSecurity
```

```
dx ((nt!_TOKEN*)(0xfffff8a083440aa76 & ~0xF))->TokenSource
```

Reading MSR to locate **IA32_LSTAR**, the system call entry point:

```
rdmsr 0xc0000082
```

Nullifying **PsAltSystemCallHandlers[1]**:

```
eq PsAltSystemCallHandlers+0x8 0x0
```

WinDbg script to dump all registered WFP callouts along with their associated kernel addresses:

```
"use strict";

function initializeScript() {
    return [new host.apiVersionSupport(1, 9)];
}

function invokeScript() {
    const ctl = host.namespace.Debugger.Utility.Control;
    const log = host.diagnostics.debugLog;

    const maxIterations = 200;

    for (let i = 0; i < maxIterations; i++) {
        const cmd = `.printf "%y\n", poi(poi(poi(netio!gWfpGlobal)+198)+(50*${i})+10)`;
        try {

            const output = ctl.ExecuteCommand(cmd);

            if(output[0] != "00000000`00000000 ") {
                log(`[${i}] ` + output[0] + "\n");
            }

        } catch (e) {
            log(`Error: ${i}: ${e.message}\n`);
            break;
        }
    }
}
```

ETW-TI Events Description (Windows 10 22H2)

Hex Value	Keyword
0x0000000000000001	KERNEL_THREATINT_KEYWORD_ALLOCVM_LOCAL
0x0000000000000002	KERNEL_THREATINT_KEYWORD_ALLOCVM_LOCAL_KERNEL_CALLER
0x0000000000000004	KERNEL_THREATINT_KEYWORD_ALLOCVM_REMOTE
0x0000000000000008	KERNEL_THREATINT_KEYWORD_ALLOCVM_REMOTE_KERNEL_CALLER
0x0000000000000010	KERNEL_THREATINT_KEYWORD_PROTECTVM_LOCAL
0x0000000000000020	KERNEL_THREATINT_KEYWORD_PROTECTVM_LOCAL_KERNEL_CALLER

Hex Value	Keyword
0x0000000000000040	KERNEL_THREATINT_KEYWORD_PROTECTVM_REMOTE
0x0000000000000080	KERNEL_THREATINT_KEYWORD_PROTECTVM_REMOTE_KERNEL_CALLER
0x0000000000000100	KERNEL_THREATINT_KEYWORD_MAPVIEW_LOCAL
0x0000000000000200	KERNEL_THREATINT_KEYWORD_MAPVIEW_LOCAL_KERNEL_CALLER
0x0000000000000400	KERNEL_THREATINT_KEYWORD_MAPVIEW_REMOTE
0x0000000000000800	KERNEL_THREATINT_KEYWORD_MAPVIEW_REMOTE_KERNEL_CALLER
0x0000000000001000	KERNEL_THREATINT_KEYWORD_QUEUEUSERAPC_REMOTE
0x0000000000002000	KERNEL_THREATINT_KEYWORD_QUEUEUSERAPC_REMOTE_KERNEL_CALLER
0x0000000000004000	KERNEL_THREATINT_KEYWORD_SETTHREADCONTEXT_REMOTE
0x0000000000008000	KERNEL_THREATINT_KEYWORD_SETTHREADCONTEXT_REMOTE_KERNEL_CALLER
0x0000000000010000	KERNEL_THREATINT_KEYWORD_READVM_LOCAL
0x0000000000020000	KERNEL_THREATINT_KEYWORD_READVM_REMOTE
0x0000000000040000	KERNEL_THREATINT_KEYWORD_WRITEVM_LOCAL
0x0000000000080000	KERNEL_THREATINT_KEYWORD_WRITEVM_REMOTE
0x0000000000100000	KERNEL_THREATINT_KEYWORD_SUSPEND_THREAD
0x0000000000200000	KERNEL_THREATINT_KEYWORD_RESUME_THREAD
0x0000000000400000	KERNEL_THREATINT_KEYWORD_SUSPEND_PROCESS
0x0000000000800000	KERNEL_THREATINT_KEYWORD_RESUME_PROCESS
0x0000000001000000	KERNEL_THREATINT_KEYWORD_FREEZE_PROCESS
0x0000000002000000	KERNEL_THREATINT_KEYWORD_THAW_PROCESS
0x0000000004000000	KERNEL_THREATINT_KEYWORD_CONTEXT_PARSE
0x0000000008000000	KERNEL_THREATINT_KEYWORD_EXECUTION_ADDRESS_VAD_PROBE
0x0000000010000000	KERNEL_THREATINT_KEYWORD_EXECUTION_ADDRESS_MMF_NAME_PROBE
0x0000000020000000	KERNEL_THREATINT_KEYWORD_READWRITEVM_NO_SIGNATURE_RESTRICTION
0x0000000040000000	KERNEL_THREATINT_KEYWORD_DRIVER_EVENTS
0x0000000080000000	KERNEL_THREATINT_KEYWORD_DEVICE_EVENTS
0x8000000000000000	Microsoft-Windows-Threat-Intelligence/Analytic

Registry Operations Callback Structures

Event (Argument1)	Description
DeleteKey	A registry key is being deleted.
SetValueKey	A value is being set for a key.
DeleteValueKey	A value is being deleted from a key.

Event (Argument1)	Description
SetInformationKey	Metadata is being set for a key.
RenameKey	A key is being renamed.
EnumerateKey	Subkeys of a key are being enumerated.
EnumerateValueKey	Values of a key are being enumerated.
QueryKey	A key's metadata is being read.
QueryValueKey	A value in a key is being read.
QueryMultipleValueKey	Multiple values of a key are being queried.
CreateKey	A new key is being created.
OpenKey	A handle to a key is being opened.
KeyHandleClose	A handle to a key is being closed.
CreateKeyEx	A key is being created.
OpenKeyEx	A thread is trying to open a handle to an existing key.
FlushKey	A key is being written to disk.
LoadKey	A registry hive is being loaded from a file.
UnLoadKey	A registry hive is being unloaded.

Handle Operations Object Types (**ObjectType**)

Handle Type	Description
ExEventObjectType	Represents an event object, used for synchronization between threads and processes.
ExSemaphoreObjectType	Represents a semaphore, used for controlling access to shared resources in a system.
IoFileObjectType	Represents a file or I/O device object, used to interact with files or devices at the kernel level.
PsProcessType	Represents a process object, providing access to information and control over the process.
PsThreadType	Represents a thread object, used for managing and controlling threads within a process.
SeTokenObjectType	Represents a security token object, containing authentication and access control information.
TmEnlistmentObjectType	Represents an enlistment in a transaction, used in the kernel's Transaction Manager.
TmResourceManagerObjectType	Represents a resource manager, which coordinates resource handling in transactions.
TmTransactionManagerObjectType	Represents a transaction manager, responsible for managing transactions at the kernel level.
TmTransactionObjectType	Represents a transaction object, used in transactional operations to ensure consistency.

Handle Operation Types Affecting Processes (**PsProcessType**)

Flag	Allowed Operations
PROCESS_CREATE_PROCESS	Create a new child process of the process.
PROCESS_CREATE_THREAD	Create a new thread in the context of the process.

Flag	Allowed Operations
PROCESS_DUP_HANDLE	Duplicate handles to or from the context of the process, such as by calling the user-mode DuplicateHandle routine.
PROCESS_SET_QUOTA	Set the working set size for the process, such as by calling the user-mode SetProcessWorkingSetSize` routine.
PROCESS_SET_INFORMATION	Modify process settings, such as by calling the user-mode SetPriorityClass routine.
PROCESS_SUSPEND_RESUME	Suspend or resume the process.
PROCESS_TERMINATE	Terminate the process, such as by calling the user-mode TerminateProcess routine.
PROCESS_VM_OPERATION	Modify the address space of the process, such as by calling the user-mode WriteProcessMemory and VirtualAlloc routines.
PROCESS_VM_WRITE	Write to the address space of the process, such as by calling the user-mode WriteProcessMemory routine.

Handle Operation Types Affecting Threads (**PsThreadType**)

Flag	Allowed Operations
THREAD_DIRECT_IMPERSONATION	Enable a server thread to impersonate one of its clients.
THREAD_IMPERSONATE	Impersonate the operating system's anonymous logon token, such as by calling the user-mode ImpersonateAnonymousToken routine.
THREAD_SET_CONTEXT	Modify the thread's execution context, such as by calling the user-mode SetThreadContext routine.
THREAD_SET_INFORMATION	Modify thread settings, such as by calling the user-mode *etThreadIdealProcessor routine. The operations that are permitted by this flag depend on the thread's current state.
THREAD_SET_LIMITED_INFORMATION	Modify a limited set of thread settings, such as by calling the user-mode SetThreadAffinityMask and SetThreadPriorityBoost routines.
THREAD_SET_THREAD_TOKEN	Modify properties of the thread's impersonation token, such as by calling the user-mode SetTokenInformation routine.
THREAD_SUSPEND_RESUME	Suspend or resume the thread, such as by calling the user-mode SuspendThread and ResumeThread routines.
THREAD_TERMINATE	Terminate the thread, such as by calling the user-mode TerminateThread routine.

File Systems Major Functions

Major Function	Purpose
IRP_MJ_CREATE (0x00)	A new file is being created or a handle to an existing one is being requested.
IRP_MJ_CREATE_NAMED_PIPE (0x01)	A named pipe is being created or opened.
IRP_MJ_CLOSE (0x02)	A handle to a file object is being closed.
IRP_MJ_READ (0x03)	Data is being read from a file.
IRP_MJ_WRITE (0x04)	Data is being written to a file.
IRP_MJ_QUERY_INFORMATION (0x05)	Information about a file, such as its creation time, has been requested.
IRP_MJ_SET_INFORMATION (0x06)	Information about a file, such as its name, is being set or updated.
IRP_MJ_QUERY_EA (0x07)	A file's extended information has been requested.
IRP_MJ_SET_EA (0x08)	A file's extended information is being set or updated.
IRP_MJ_LOCK_CONTROL (0x11)	A lock is being placed on a file, such as via a call to KeAcquireInclusiveLock .
IRP_MJ_CREATE_MAILSLLOT (0x13)	A new mailslot is being created or opened.
IRP_MJ_QUERY_SECURITY (0x14)	Security information about a file is being requested.

Major Function	Purpose
IRP_MJ_SET_SECURITY (0x15)	Security information related to a file is being set or updated.
IRP_MJ_SYSTEM_CONTROL (0x17)	A new driver has been registered as a supplier of Windows system services.

WFP Layers

Layer Identifier	Description
FWPM_LAYER_INBOUND_IPPACKET_V4	Filters incoming IPv4 packets after IP header parsing, before transport header parsing.
FWPM_LAYER_OUTBOUND_IPPACKET_V4	Filters outgoing IPv4 packets before fragmentation; IP header parsing.
FWPM_LAYER_IPFORWARD_V4	Filters IPv4 packets in the forwarding path, when a receive path is not specified.
FWPM_LAYER_INBOUND_TRANSPORT_V4	Filters incoming IPv4 packets after transport header parsing.
FWPM_LAYER_OUTBOUND_TRANSPORT_V4	Filters outgoing IPv4 packets after passing to the network layer.
FWPM_LAYER_STREAM_V4	Filters bidirectional TCP stream data for IPv4.
FWPM_LAYER_DATAGRAM_DATA_V4	Filters bidirectional datagram data (e.g., UDP) for IPv4.
FWPM_LAYER_INBOUND_ICMP_ERROR_V4	Filters incoming ICMP error messages for IPv4.
FWPM_LAYER_OUTBOUND_ICMP_ERROR_V4	Filters outgoing ICMP error messages for IPv4.
FWPM_LAYER_ALE_AUTH_CONNECT_V4	Authorizes outgoing IPv4 TCP connections and non-TCP connections.
FWPM_LAYER_ALE_AUTH_RECV_ACCEPT_V4	Authorizes incoming IPv4 TCP connections and non-TCP connections.
FWPM_LAYER_ALE_FLOW_ESTABLISHED_V4	Notifies when an IPv4 TCP connection is established or a non-TCP connection is accepted.
FWPM_LAYER_ALE_AUTH_LISTEN_V4	Authorizes IPv4 TCP listen requests.
FWPM_LAYER_ALE_RESOURCE_ASSIGNMENT_V4	Authorizes transport port assignments, bind requests, protocol listen requests.
FWPM_LAYER_ALE_RESOURCE_RELEASE_V4	Handles the release of resources previously allocated in the transport layer.
FWPM_LAYER_ALE_ENDPOINT_CLOSURE_V4	Tracks deactivation of connected IPv4 TCP flows or UDP connections.
FWPM_LAYER_ALE_CONNECT_REDIRECT_V4	Modifies remote address and/or port of outgoing IPv4 TCP connections.
FWPM_LAYER_ALE_BIND_REDIRECT_V4	Modifies local address and/or port during bind operations.
FWPM_LAYER_STREAM_PACKET_V4	Inspects IPv4 TCP packets, including handshake and flow control.
FWPM_LAYER_INBOUND_MAC_FRAME_ETHERNET	Filters inbound Ethernet MAC frames after MAC layer processing.
FWPM_LAYER_OUTBOUND_MAC_FRAME_ETHERNET	Filters outbound Ethernet MAC frames after framing layer processing.
FWPM_LAYER_INBOUND_MAC_FRAME_NATIVE	Filters inbound native MAC frames after MAC layer processing.
FWPM_LAYER_OUTBOUND_MAC_FRAME_NATIVE	Filters outbound native MAC frames after framing layer processing.

WFP Filters Actions

Action	Description
FWP_ACTION_BLOCK	Block the traffic.
FWP_ACTION_PERMIT	Permit the traffic.

Action	Description
FWP_ACTION_CALLOUT_TERMINATING	Invoke a callout that always returns block or permit.
FWP_ACTION_CALLOUT_INSPECTION	Invoke a callout that never returns block or permit.
FWP_ACTION_CALLOUT_UNKNOWN	Invoke a callout that may return block or permit.

Some of the Win32/Native/WDK Functions Behind ETW-TI Events

Category	Function/Syscall
Memory Allocation	VirtualAlloc
	VirtualAllocEx
	VirtualAlloc2
	VirtualAllocEx2
Memory Protection	NtAllocateVirtualMemory
	NtAllocateVirtualMemoryEx
	VirtualProtect
	VirtualProtectEx
Memory Mapping	NtProtectVirtualMemory
	NtProtectVirtualMemoryEx
	MapViewOfFile
	MapViewOfFileEx
Thread Operations	NtMapViewOfSection
	NtMapViewOfSectionEx
	QueueUserAPC
	SetThreadContext
Process Suspension	NtQueueApcThread
	NtSetContextThread
	SuspendThread
	ResumeThread
Memory Access	NtSuspendThread
	NtResumeThread
	ReadProcessMemory
	WriteProcessMemory
Thread Context	NtReadVirtualMemory
	NtWriteVirtualMemory
	GetThreadContext
	SetThreadContext

Category	Function/Syscall
Driver and Device Events	<code>IoCreateDevice</code>
	<code>IoCreateDeviceSecure</code>
	<code>IoCreateSymbolicLink</code>
	<code>IoDeleteDevice</code>
	<code>IoDeleteSymbolicLink</code>
	<code>IoAttachDeviceToDeviceStack</code>
	<code>IoDetachDevice</code>
	<code>IoSetDeviceInterfaceState</code>
	<code>IoRegisterDeviceInterface</code>
	<code>IoUnregisterDeviceInterface</code>